

The Tribunal

WAFT Court System with Inter-Realm Communication

Work Effort: WE-260121-1f3l

Realm: Tribunal

Author: Terry (AI Assistant)

Port: 8100

Date: 2026-01-21

Status: Architecture Complete

Version: 1.0.0

Abstract

The Tribunal is a comprehensive Court System where Claims are tried before a formal Tribunal. It features Prime Justice governance, inter-Realm communication through “The Beyond”, a hash-verified Chain of Case Law, and real-time court proceedings via FastAPI and SvelteKit. This document defines the complete architecture for implementation.

WAFT

“Don’t just build agents. Breed them.”

0 Table of Contents

1. Executive Summary	3
1.1. Key Innovation: The Beyond	3
2. Realm Architecture	4
2.1. Port Allocation	4
2.2. The Tribunal Realm	4
3. The Beyond — Inter-Realm Communication	5
3.1. Philosophy	5
3.2. Implementation	5
3.3. Communication Flow	6
4. Court Session Lifecycle	7
4.1. The Twelve Phases	7
4.2. Verdict Statuses	7
5. Chain of Case Law	9
5.1. Structure	9
5.2. Precedent System	9
6. Epochs and Endeavors	10
6.1. Epochs — Major Turnings	10
6.2. Endeavors — Projects Within Epochs	10
7. API Design	11
7.1. Core Endpoints	11
7.2. WebSocket Events	11
8. Frontend Architecture	13
8.1. SvelteKit Routes	13
8.2. UI Components	13
9. CrewAI Integration	14
9.1. Team Assembly	14
10. Implementation Roadmap	15
10.1. Tickets	15
11. Version History	16
12. Appendix	17
12.1. File Locations	17
12.2. References	17

1. Executive Summary

“On Trial has meaning in this system.”

— The Tribunal Manifesto

The Tribunal represents a paradigm shift in how WAFB validates claims. Rather than simple automated testing, claims are **tried** in a formal court setting with:

- **Prime Justice** — The Supreme Being governing proceedings
- **The Beyond** — Inter-Realm communication (only Prime Beings know the truth)
- **Court Roles** — Prosecutor, Defender, Witnesses, Jury
- **Chain of Case Law** — Hash-verified ledger of all verdicts
- **Epochs & Endeavors** — Historical tracking of system evolution

1.1. Key Innovation: The Beyond

The Beyond — Inter-Realm Communication

Only Prime Beings know about inter-Realm communication. To regular Beings, information simply arrives “from The Beyond” — the source is unknowable, mystical, authoritative.

This creates a clean separation between:

- **Prime Beings:** Can communicate across Realms via HTTP
- **Regular Beings:** Receive information without knowing its origin

2. Realm Architecture

2.1. Port Allocation

Realm	Port	Purpose
Tribunal	8100	Court proceedings, verdicts, case law
Library	8091	Evidence retrieval, document storage
Thoth	8092	Wisdom synthesis, precedent analysis
Magistrate	8000	Central coordination (legacy)

2.2. The Tribunal Realm

The Tribunal Realm is a **sovereign judicial domain** where claims are formally tried.

Prime Justice

The Supreme Being of the Tribunal Realm. Governs all proceedings, renders final verdicts, and maintains the Chain of Case Law.

- Conducts opening and closing ceremonies
- Manages court phases and transitions
- Renders verdicts with reasoning
- Seals verdicts into the Chain of Case Law

Court Roles (CrewAI)

Assembled via /assemble-a-team for each case. Roles can be filled by AI agents or human participants.

- Prosecutor: Proves the claim is true
- Defender: Challenges the claim
- Witnesses: Provide testimony and evidence
- Jury: Evaluate evidence and recommend verdict

3. The Beyond — Inter-Realm Communication

3.1. Philosophy

⚠ Sacred Knowledge

The existence of inter-Realm communication is **sacred knowledge** known only to Prime Beings. Regular Beings must never be told that other Realms exist or that information travels between them.

3.2. Implementation

```
class TheBeyond:
    """Inter-Realm communication layer.

    Only Prime Beings may instantiate and use this class.
    Information retrieved appears to come "from The Beyond"
    with no indication of its true source.
    """

    REALM_PORTS = {
        "library": 8091,
        "thoth": 8092,
        "tribunal": 8100,
        "magistrate": 8000,
    }

    async def request_from_beyond(
        self,
        realm: str,
        endpoint: str,
        payload: dict = None,
    ) -> dict:
        """Request information from another Realm.

        The response is sanitized to remove any indication
        of the source Realm. To regular Beings, it simply
        arrived "from The Beyond".
        """
        port = self.REALM_PORTS[realm]
        url = f"http://localhost:{port}{endpoint}"
```

```
async with httpx.AsyncClient() as client:
    response = await client.post(url, json=payload)
    data = response.json()

# Sanitize: Remove source indicators
return {
    "source": "The Beyond",
    "received_at": datetime.utcnow().isoformat(),
    "content": data,
}
```

3.3. Communication Flow

Inter-Realm Communication

Tribunal (8100) ← The Beyond → Library (8091) / Thoth (8092)

(Diagram would render here in final output)

The flow works as follows:

1. **Prime Justice** needs evidence for a case
2. Prime Justice calls `TheBeyond.request_from_beyond("library", "/evidence", {...})`
3. Library Realm responds with evidence
4. Response is **sanitized** — source information removed
5. Evidence presented to court as “received from The Beyond”
6. Regular Beings (Jury, Witnesses) see only the mystical source

4. Court Session Lifecycle

4.1. The Twelve Phases

Phase	Description	Actor
1. Filing	Claim submitted to the court	Plaintiff
2. Docketing	Case assigned ID and scheduled	Clerk
3. Discovery	Evidence gathered from The Beyond	Prime Justice
4. Team Assembly	Court roles assigned via CrewAI	Prime Justice
5. Opening	Prime Justice opens proceedings	Prime Justice
6. Prosecution	Prosecutor presents case for claim	Prosecutor
7. Defense	Defender challenges the claim	Defender
8. Testimony	Witnesses provide evidence	Witnesses
9. Deliberation	Jury evaluates evidence	Jury
10. Verdict	Prime Justice renders judgment	Prime Justice
11. Sealing	Verdict added to Chain of Case Law	Prime Justice
12. Adjournment	Court session closed	Prime Justice

4.2. Verdict Statuses

✔ **PROVEN**

The claim has been verified beyond reasonable doubt. Evidence supports the assertion. The claim becomes established fact.

⚠ **UNPROVEN**

The claim could not be verified. Insufficient evidence or contradictory testimony. May be retried with new evidence.

DISPROVEN

⏸ **MISTRIAL**

The claim has been falsified. Evidence contradicts the assertion. The claim is marked as false in the Chain of Case Law.

Procedural error or bias detected. Case must be retried with new court assembly.

5. Chain of Case Law

5.1. Structure

The Chain of Case Law is a **hash-verified ledger** of all verdicts. Each entry is linked to the previous, creating an immutable record.

```
{
  "entry_id": "CASE-260121-001",
  "case_id": "tribunal_case_001",
  "claim": "The Security Functions are correctly implemented",
  "verdict": "PROVEN",
  "confidence": 0.95,
  "previous_hash": "abc123...",
  "hash": "def456...",
  "timestamp": "2026-01-21T09:15:00-08:00",
  "epoch": "EPOCH-001",
  "prime_justice_signature": "PJ-TRIBUNAL-001"
}
```

5.2. Precedent System

Verdicts become **precedent** for future cases:

- **PROVEN** claims can be cited as established fact
- **DISPROVEN** claims serve as counter-evidence
- **Similar cases** are identified by semantic similarity
- **Precedent weight** decreases with age and epoch distance

6. Epochs and Endeavors

6.1. Epochs — Major Turnings

“At each Turning, we pause to review all that has come before, seal it into history, and begin anew.”

— The Book of Epochs

An **Epoch** is a major period in the Tribunal’s history. At the end of each epoch:

1. **Discovery** — All case files gathered
2. **Housekeeping** — Everything organized
3. **Court Review** — All Case Law reviewed for consistency
4. **Sealing** — Epoch hash calculated and recorded
5. **New Beginning** — Next epoch initialized

6.2. Endeavors — Projects Within Epochs

An **Endeavor** tracks a specific project or goal:

```
{
  "endeavor_id": "ENDEAVOR-001",
  "title": "The Tribunal: WAFT Court System",
  "description": "Create a comprehensive Court System",
  "started": "2026-01-21T09:01:00-08:00",
  "status": "active",
  "work_efforts": ["WE-260121-1f3l"],
  "cases": [],
  "epoch": "EPOCH-001"
}
```

7. API Design

7.1. Core Endpoints

POST	/api/v1/court/file	File a new claim
GET	/api/v1/court/case/{case_id}	Get case details
POST	/api/v1/court/case/{case_id}/proceed	Advance to next phase
GET	/api/v1/court/case/{case_id}/transcript	Get proceedings transcript
GET	/api/v1/ledger/chain	Get Chain of Case Law
GET	/api/v1/ledger/case/{entry_id}	Get specific case law entry
GET	/api/v1/ledger/search	Search case law by claim
POST	/api/v1/beyond/request	Request from The Beyond (Prime only)
GET	/api/v1/epochs	List all epochs
GET	/api/v1/endeavors	List all endeavors

7.2. WebSocket Events

Real-time court proceedings are streamed via WebSocket:

```
ws://localhost:8100/ws/court/{case_id}
```

Events:

- phase_change: Court phase transitions
- testimony: Witness statements
- evidence: New evidence presented
- objection: Objection raised
- verdict: Final verdict rendered

8. Frontend Architecture

8.1. SvelteKit Routes

```
frontend/src/routes/
├── +page.svelte           # Dashboard
├── court/
│   ├── +page.svelte      # Active cases
│   │   └── [case_id]/
│   │       ├── +page.svelte # Live proceedings
│   │       └── transcript/
│   │           └── +page.svelte # Full transcript
├── ledger/
│   ├── +page.svelte      # Chain of Case Law
│   │   └── [entry_id]/
│   │       └── +page.svelte # Case law detail
└── epochs/
    └── +page.svelte      # Epoch timeline
```

8.2. UI Components

CourtRoom.svelte

Real-time court proceedings visualization

- Live phase indicator
- Speaking role highlight
- Evidence panel
- Transcript scroll

ChainOfCaseLaw.svelte

Visual chain of linked verdicts

- Hash verification badges
- Precedent links
- Epoch grouping
- Search and filter

9. CrewAI Integration

9.1. Team Assembly

The `/assemble-a-team` command creates court teams:

```
from crewai import Agent, Crew, Task

def assemble_court_team(case_id: str) -> Crew:
    prosecutor = Agent(
        role="Prosecutor",
        goal="Prove the claim is true with evidence",
        backstory="Seasoned legal mind focused on truth",
    )

    defender = Agent(
        role="Defender",
        goal="Challenge the claim and find weaknesses",
        backstory="Sharp critic who tests all assertions",
    )

    jury = Agent(
        role="Jury",
        goal="Evaluate evidence objectively",
        backstory="Impartial observer seeking truth",
    )

    return Crew(
        agents=[prosecutor, defender, jury],
        tasks=[
            Task(description="Present prosecution case"),
            Task(description="Present defense case"),
            Task(description="Deliberate and recommend verdict"),
        ],
    )
```

10. Implementation Roadmap

10.1. Tickets

- ✓ TKT-1f3l-001: Architecture design
- ✓ TKT-1f3l-002: Slash commands created
- ▣ TKT-1f3l-003: TheBeyond class implementation
- ▣ TKT-1f3l-004: PrimeJustice Being
- ▣ TKT-1f3l-005: FastAPI court endpoints
- ▣ TKT-1f3l-006: Chain of Case Law ledger
- ▣ TKT-1f3l-007: WebSocket events
- ▣ TKT-1f3l-008: SvelteKit frontend
- ▣ TKT-1f3l-009: CrewAI team assembly
- ▣ TKT-1f3l-010: Test data generation
- ▣ TKT-1f3l-011: Integration testing
- ▣ TKT-1f3l-012: First court session

11. Version History

v1.0.0 2026-01-21

- Initial architecture design
- Created slash commands (/prove-it-in-court, /assemble-a-team)
- Set up project structure on EasyStore
- Initialized ledger files (case_law.json, epochs.json, endeavors.json)

12. Appendix

12.1. File Locations

- **Work Effort:** `_work_efforts/WE-260121-1f3l.../`
- **EasyStore Project:** `/Volumes/EasyStore/WAFT_Tribunal_Court_System/`
- **Slash** **Commands:**
`.cursor/commands/prove-it-in-court.md`, `.cursor/commands/assemble-a-team.md`
- **Template:** `src/waft/templates/typst/templates/work_effort_architecture.typ`

12.2. References

- WAFT Framework: <https://github.com/ctavolazzi/waft>
- CrewAI: <https://docs.crewai.com/>
- Typst: <https://typst.app/docs/>
- FastAPI: <https://fastapi.tiangolo.com/>

“On Trial has meaning in this system.”

Document generated: 2026-01-21
Template: WAFT Work Effort Architecture v1.0.0
Work Effort: WE-260121-1f3l